



1

Développement des applications Android

Option 1 : XML Views



Option 2 : Jetpack Compose



1

Jetpack Compose vs. XML Views

2

- En 2025, **Jetpack Compose** est devenu l'outil privilégié pour les nouvelles applications Android, tandis que **XML Views** reste très présent dans les projets existants.
- D'après les analyses récentes :
 - **Compose** est désormais le standard recommandé par Google et il est activement développé, avec un écosystème de bibliothèques qui s'étoffe rapidement.
La majorité des nouveaux projets démarrent directement en Compose, car il permet un développement plus rapide, un code plus concis et une meilleure gestion de l'état.
 - **XML Views** reste massivement utilisé dans les applications déjà en production ou dans les équipes qui maintiennent de gros projets historiques, car migrer vers Compose demande du temps et des ressources.

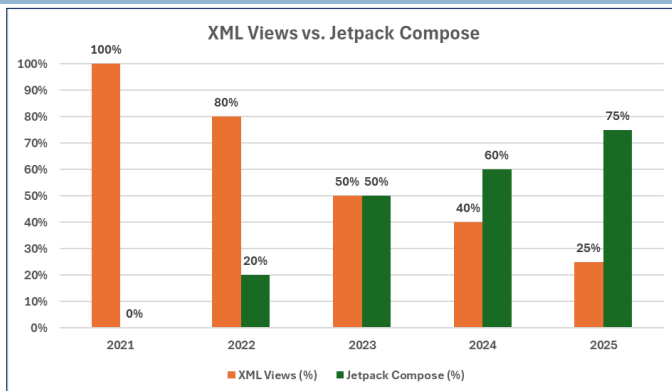
Nouveaux projets → Compose domine.

Projets existants → XML reste majoritaire.

2

Jetpack Compose vs. XML Views

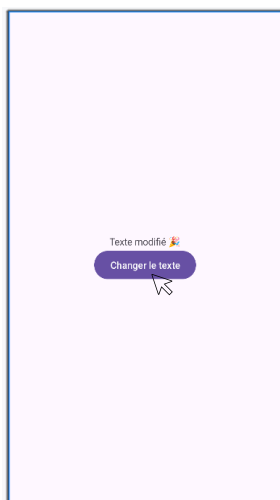
3



- En **2021**, **XML** dominait presque totalement (Compose inexistant).
- Vers **2023**, les deux approches étaient utilisées à parts égales.
- En **2025**, **Compose** est largement en tête, autour de **75 %** d'adoption dans les nouveaux projets, tandis que **XML** tombe à environ **25 %**.

3

Mini - Exemple

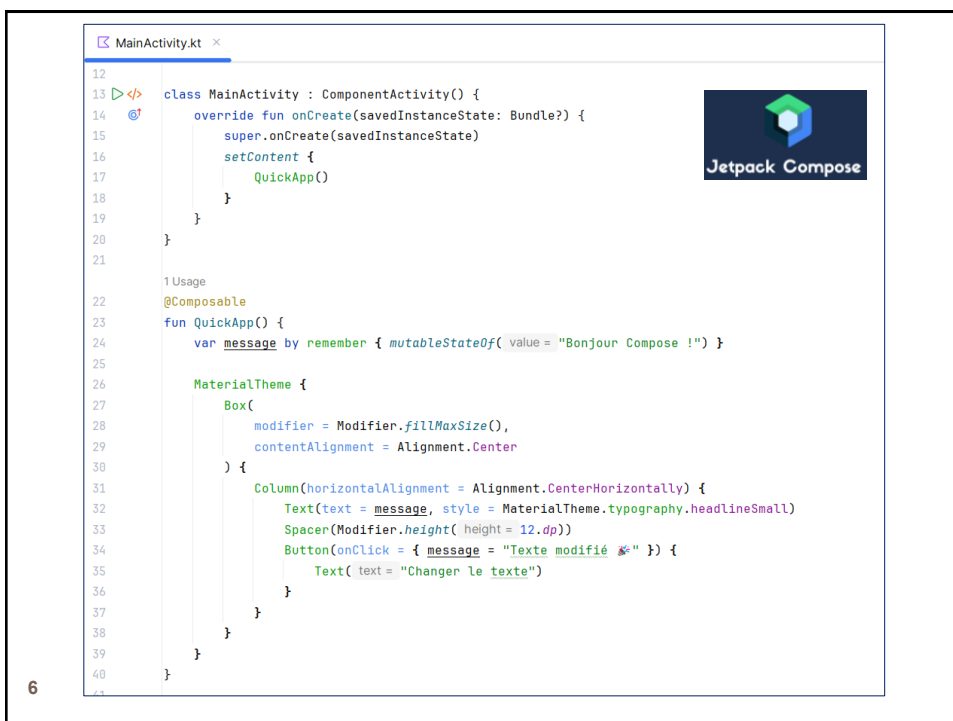


4

4



5



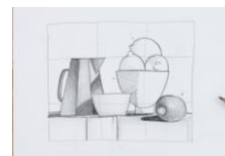
6

Jetpack Compose vs. XML Views

- **XML Views**, c'est comme construire avec des briques  bien rangées.



- **Jetpack Compose**, c'est comme dessiner librement  avec des formes dynamiques.



→ Deux styles, deux philosophies... une même ambition : créer des interfaces efficaces et élégantes.

7



8

Chapitre 3

Options de Développement Android

« Option 1 : XML Views »



Dorra DHAOU

Extrait du cours de : **Dr. Naoufel KHAYATI**

AU. 2026 - 2027

8

PLAN

9

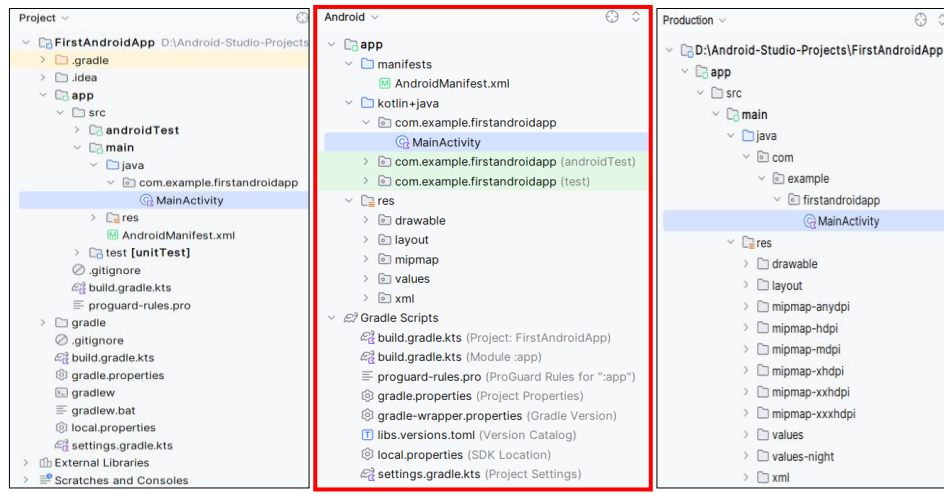
- Structure d'un projet Android à base d'XML Views
 - ▣ Le dossier *java*
 - ▣ Le dossier *res*
 - ▣ Le dossier *manifests*
 - ▣ Les scripts *Gradle*
- Ressources
 - ▣ La classe *R*
 - ▣ Organisation

9

Structure d'un projet

10

- Différentes vues d'un projet (Project, Android, Production, Packages, etc.)

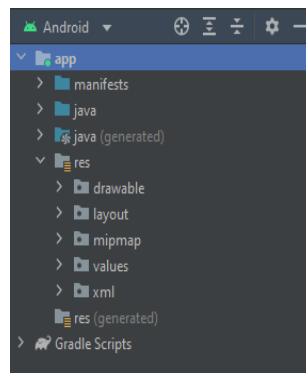


10

Structure d'un projet

11

- **manifest** : la carte d'identité de l'application, et permet d'autoriser l'exécution des activités et autres actions de l'application.
- **java** : contenant l'ensemble du code source Java ou Kotlin de l'application MainActivity.java
- **res** : contenant toutes les ressources telles que les images, les vues de l'interface graphique, etc., nécessaires à l'application.
- **gradle** : sert dans la configuration pour le nouveau moteur de production. Il sera utilisé pour construire notre application afin de la déployer sur notre smartphone ou sur le Play Store.



11

Structure d'un projet – Le dossier *java*

12

- **java** : contient le paquetage de l'application et ceux des tests.
 - ▢ Le paquetage de l'application contient les fichiers sources .kt des activités (ici, un seul fichier MainActivity.kt)

```

package com.example.firstandroidapp

import android.os.Bundle
import androidx.activity.enableEdgeToEdge
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView(R.layout.activity_main)
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main)) { v, insets ->
            val systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars())
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom)
            insets
        }
    }
}

```



12

Structure d'un projet – Le dossier res

13

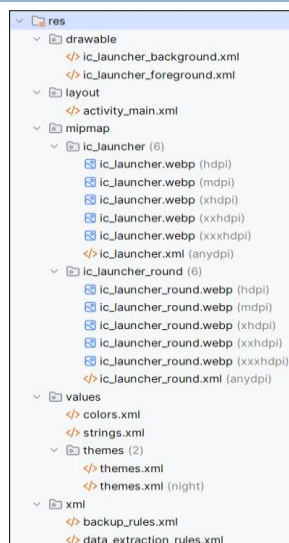
- **res** : contient les ressources de l'application, càd les éléments qui s'afficheront à l'écran ou qui influenceront ce qui s'affichera à l'écran.
- Les ressources sont des éléments capitaux dans une application Android.
- On y trouve par exemple des chaînes de caractères, des couleurs, des tailles, des images, des fichiers XML de layout, des fichiers XML de menus, etc.

13

Structure d'un projet – Le dossier res

14

- Les ressources sont de plusieurs types.
- Pour permettre à Android de les retrouver facilement, chaque type de ressources est associé à un répertoire particulier.
- Aucune ressource ne doit être placée directement dans res/
 - Erreur de compilation



14

Structure d'un projet — Le dossier res

15

- **res/drawable:** Dessins et images
 - On y trouve les images matricielles (PNG, JPEG, GIF).
 - On peut y trouver aussi des fichiers XML dont le contenu décrit des formes ou des dessins.
- **res/mipmap:** Images aussi, qui servent comme des icônes, par exemple.
 - Une image placée dedans, se verra avoir différentes densités (mdpi, hdpi, xhdpi, etc.) pour s'adapter à différentes résolutions d'écrans.

15

Structure d'un projet — Le dossier res

16

- **res/layout:** Mise en page d'écran ou interface graphique (gabarit)
 - Les fichiers XML qui représentent la disposition que doivent adopter les vues.
- **res/menu** (n'apparaît pas dans notre exemple): Pour les menus
 - Les fichiers XML décrivant des menus.
- **res/values:** Données
 - Les ressources qui appartiennent à cette catégorie sont nombreuses. On y trouve entre autres des chaînes de caractères, des dimensions, des couleurs, etc.
- **res/xml:** Un emplacement polyvalent pour divers fichiers XML utilisés pour la configuration, les préférences, les animations et toutes autres ressources personnalisées.

16

Structure d'un projet — Le dossier *manifests*

17

- **manifests:** Contient le fichier **AndroidManifest.xml** qui est généré dès la création du projet.
 - Il ne doit être ni renommé ni déplacé.
- Ce fichier est lu au lancement de l'application par le système pour identifier le comportement de votre application.
- Il déclare et décrit l'ensemble des éléments de l'application:
 - Nom et package de l'application, son icône, thème, etc.
 - Activités, Services, Intents
 - le point d'entrée de l'application
 - quels composants constituent ce programme;
 - Permissions nécessaires à l'exécution du programme
 - Accès à Internet, accès à l'appareil photo, appels téléphoniques, etc.

17

Structure d'un projet — Le dossier *manifests*

18

□ Le fichier **AndroidManifest.xml**

```

1  <?xml version="1.0" encoding="utf-8"?>
2  <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3          xmlns:tools="http://schemas.android.com/tools">
4
5      <application
6          android:allowBackup="true"
7          android:dataExtractionRules="@xml/data_extraction_rules"
8          android:fullBackupContent="@xml/backup_rules"
9          android:icon="@mipmap/ic_launcher"
10         android:label="@string/first_android_app"
11         android:roundIcon="@mipmap/ic_launcher_round"
12         android:supportRtl="true"
13         android:theme="@style/Theme.FirstAndroidApp"
14         tools:targetApi="31">
15         <activity
16             android:name=".MainActivity"
17             android:exported="true">
18             <intent-filter>
19                 <action android:name="android.intent.action.MAIN" />
20
21                 <category android:name="android.intent.category.LAUNCHER" />
22             </intent-filter>
23         </activity>
24     </application>
25
26 </manifest>

```

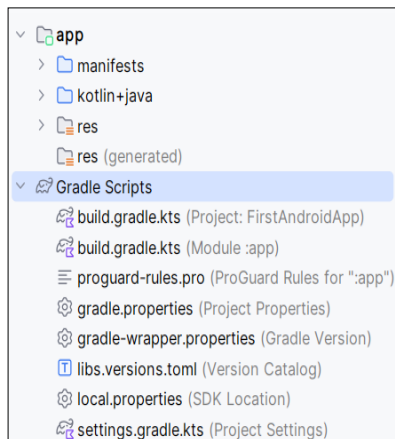
18

Structure d'un projet — Les scripts *Gradle*

19

□ Gradle Scripts

- Android Studio se base sur le système de build «GRADLE ».
- Outil de génération de projets comme Make, Ant et Maven.
- De même que Make se sert d'un fichier Makefile, Gradle se sert d'un fichier nommé build.gradle.



19

Structure d'un projet — Les scripts *Gradle*

20

- Sous Android Studio, il y a deux build.gradle : (1)
 - Le premier script dans le dossier racine du projet.
 - ➔ build.gradle (Project)
 - Il sert à indiquer la configuration commune à tous les modules du projet (dépendances générales, plugins généraux, etc.).

```
// Top-level build file where you can add configuration options common to all sub-projects/modules.
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.kotlin.android) apply false
}
```

20

Structure d'un projet — Les scripts *Gradle*

21

- Sous Android Studio, il y a deux build.gradle : (2)
 - ▣ Le deuxième script dans le dossier 'app' - build.gradle (Module/App) - pour la configuration du module (configuration de la compilation, dépendances et plugins spécifiques).

```

plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.android)
}

android {
    namespace = "com.example.firstandroidapp"
    compileSdk = 34

    defaultConfig {
        applicationId = "com.example.firstandroidapp"
        minSdk = 26
        targetSdk = 34
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile( name: "proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_1_8
        targetCompatibility = JavaVersion.VERSION_1_8
    }
}

```

21

Structure d'un projet — Les scripts *Gradle*

22

- Généralement, ces deux fichiers build.gradle peuvent être mis à jour dans les cas suivants :
 - ▣ Changer la version de l'API Android de développement
 - ▣ Changer la version de l'outil de compilation
 - ▣ Changer la version minimale de l'API supportée par l'application
 - ▣ Appeler des bibliothèques externes
 - ▣ Etc.

22

Ressources – La classe R

23

- Comme Android est destiné à être utilisé sur une grande variété de supports, il fallait trouver une solution pour :
 - ▢ Permettre à une application de s'afficher de la même manière sur différentes tailles et résolutions d'écrans.
 - ▢ Faire en sorte que les textes s'adaptent à la langue de l'utilisateur.
- Pour cela, Android organise les différentes ressources d'une application d'une manière très précise, de façon qu'il sache quels éléments utiliser pour quels types de terminaux.
- Ces ressources sont utilisées dans le code à travers la classe R.
 - ▢ Android régénère automatiquement cette classe à chaque changement dans le projet.
 - ▢ Toutes les ressources sont accessibles à travers R, dès qu'elles sont déclarées dans le fichier XML ou que le fichier associé est déposé dans le dossier adéquat.

23

Ressources – La classe R

24

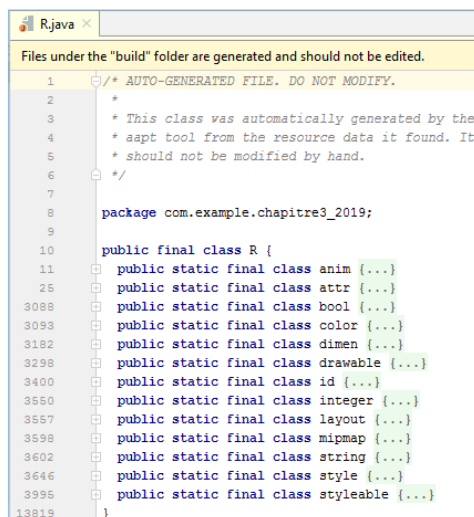
- Pour les versions Android Studio 3.6 et plus, AGP (Android Gradle Plugin) via son AAPT2 génère désormais directement le bytecode de la classe R, au lieu du fichier R.java.
 - ▢ On ne trouvera plus le fichier R.java dans l'arborescence du projet.
 - ▢ Avec les dernières versions d'Android Studio + Gradle, Google ne génère plus systématiquement un fichier R.java lisible. À la place, la classe R est compilée directement dans un fichier .jar (souvent nommé R.jar) placé dans :
`app/build/intermediates/compile_and_runtime_not_namespaced_r_class_jar/debug/R.jar`
- **NB : les quelques slides suivants vous donneront une idée sur cette classe ainsi que sur ses classes internes afin de comprendre l'utilisation des ressources qui y sont déclarées.**

24

Ressources – La classe R

25

- La classe R contient des **sous-classes statiques** pour chaque type de ressources défini dans le projet (ex. : R.layout, R.string, R.drawable, etc.)..
- Chaque sous-classe correspond à un type de ressource.
- À l'intérieur, on trouve des **constantes entières** (IDs) qui représentent chaque ressource déclarée.
- Pour utiliser une ressource, il faut connaître son **type** et son **nom** (identifiant).



```

1  /* AUTO-GENERATED FILE. DO NOT MODIFY.
2
3  * This class was automatically generated by the
4  * aapt tool from the resource data it found. It
5  * should not be modified by hand.
6  */
7
8  package com.example.chapitre3_2019;
9
10 public final class R {
11     public static final class anim {...}
12     public static final class attr {...}
13     public static final class bool {...}
14     public static final class color {...}
15     public static final class dimen {...}
16     public static final class drawable {...}
17     public static final class id {...}
18     public static final class integer {...}
19     public static final class layout {...}
20     public static final class mipmap {...}
21     public static final class string {...}
22     public static final class style {...}
23     public static final class styleable {...}
24 }

```

25

Ressources – La classe R

26

La classe « layout »

- Une classe **interne**, pour y accéder, il faut faire référence à la classe qui la contient (R).

```

public static final class layout {
    public static final int activity_main=0x7f09001c;
}

```

- Le mot clé **static**, dans le cas d'une classe interne, signifie que la classe n'est pas liée à une instanciation de la classe qui l'encapsule.
 - ▢ Pour accéder à **layout**, on ne doit pas nécessairement créer un objet de type **R**.
 - ▢ On peut y accéder par **R.layout**
 - ➔ Comme elle est **static**, on peut l'appeler directement : **R.layout.activity_main** sans créer d'objet R.
- Le mot clé **final** signifie que l'on ne peut pas créer de classe dérivée de layout (on ne peut pas en faire une sous-classe).

26

Ressources – La classe R

27

- Dans **R.layout**, chaque ressource est représentée par une constante entière (public static final int).
 - Cet entier est unique et correspond au nom du fichier XML. Exemple : `activity_main.xml` → devient une constante entière `R.layout.activity_main`.
 - Ces entiers sont en réalité des codes hexadécimaux sur **32 bits**, générés automatiquement.
 - Cet entier a le même nom qu'un fichier de ressources, tout simplement parce qu'il représente ce fichier XML.
 - Cet entier est un identifiant unique pour le fichier de mise en page qui s'appelle `activity_main.xml`.
 - L'identifiant d'un layout est accessible comme suit:

Java / Kotlin

`R.layout.activity_main`

XML

`"@layout/activity_main"`

27